

GRID CONTROL — UNIFIED BUILD PLAN · FINAL (Jun 9 2026)

Merges: this doc's Jun 9 foundation plan + [Monday_Build_Plan_2026-06-08.pdf](#) (Platform Growth Dept) + **FLOW v2 client-experience decisions** ([docs/FLOW_V2_DECISIONS.md](#)) + **Brand-Book report spec** ([docs/BRAND_BOOK_REPORT_SPEC.md](#)). This is the FINAL consolidated plan. Branch: [gridcontrol-rebuild](#) . Scope core: **UC1**. Resume safe word: [GRIDLOCK-DASHBOARD-VISION-09JUN](#) . Status: **PLAN ONLY — build not started**. Execute on explicit "build" / "go". **Validation goal:** prove the rebuilt pipeline end-to-end on TWO brands — **offgrid-creatives-ai = NEW-brand path**, **askgauravai = EXISTING-brand path** (see §4.5 Pilot). Merge rule (Jun 8 vision): **foundations BEFORE more agents**. → Wave 1 = foundations; Wave 1.5 = client-experience + brand-book layer; Wave 2 = growth agents; Shelf = Phase 2. **Dashboard front-end is OUT of this plan** (Gaurav, Jun 9): finish the SYSTEM here first, THEN rebuild the dashboard with Emergent/Lovable on top of it. Design exploration + cockpit-as-home decision parked in [docs/DASHBOARD_DESIGN_EXPLORATION.md](#) . So Wave 1 here = backend/data/API layer; the new UI is a later phase.

0. HOW THE TWO PLANS RELATE

- **Today (Jun 9)** = the *foundation / brain* layer: persistent memory, cost wiring, daily surface, model gateway, scheduler.
- **Monday (Jun 8)** = the *Platform Growth Department* (CM, DM Hunter, funnel, email) + tooling (Odysseus M7, AI Setter/Postiz/Obsidian M8).
- They sequence, they don't compete. Agents WORK but had no brain → build the brain first, then the growth agents on top.

1. CONFLICTS THE NEW SYSTEM SETTLES (Monday decisions overturned)

#	Monday said	New system	Resolution & why
C1 Memory	Obsidian vault = ONE memory + ChromaDB local vector	Supabase thin + pgvector	Supabase wins — Obsidian/Chroma are local-on-Mac; break when agents run server-side for a hosted client.
C2 Dev notes	Obsidian for Gaurav+Claude shared memory	memory/MEMORY.md + context packages	Drop Obsidian — third surface = the "systems to forget" trap. Keep the one we hook-load.
C3 Scheduler	crontab + Postiz for publish	APScheduler	APScheduler = agent-run cadence (replaces buggy crontab). Publish cadence keeps publishing/* ; Postiz → Phase 2.
C4 Gateway	M7#4 Ollama/Cookbook offload	LiteLLM	Merged: LiteLLM gateway <i>with Ollama local tier</i> . M8.6 (\$20 Pro, not Max) reinforces \$0-local grunt.

Dropped: Obsidian-as-memory, ChromaDB ([src/memory_vector.py](#) , [chroma_client.py](#)). Graphify stays (disposable analyzer, not memory).

2. ALREADY DONE — DO NOT REBUILD (both status sections)

-  Triggers: [/api/agents/run](#) , [/api/pipeline/daily-run](#) , subprocess + Managed-Agents
-  Publishers: IG/LinkedIn/X/YouTube in [publishing/](#) + [publish_runner.py](#)
-  Onboarding: 3-step wizard + intake form + [scripts/onboard_brand.py](#) + [docs/BRAND_ONBOARDING.md](#)
-  Supabase tables: [agent_runs](#) , [agent_outputs](#) , [session_state](#) , [brand_memory](#) , [usage_logs](#) , RLS
-  M0 voice restore ([askgauravai/voice_profile.json](#))
-  M4 insights: pivoted to Instagram Login API → [agents/meta_insights.py](#) wired into [data_analyst.py](#) (FB-Page path reverted)
-  M6 offgrid-creatives-ai onboarded (IG + LinkedIn + YouTube; X manual) — = the **NEW-brand pilot target (§4.5)**; profile re-anchored to Reporting SaaS
-  Memory hooks: [base_agent.py](#) session_start/save/end + [session_state.json](#)
-  Cost infra: [cost_tracker.py](#) + [tracing.py](#) (not reliably stored per run → ₹0 bug)
-  Data Analyst + [meta_insights.py](#) (needs binding to real metrics)

2.5 MODEL & EFFORT ROUTING

Constraint: Claude Code cannot self-switch its model mid-session; `/model` is Gaurav's control. So an unattended overnight build runs on ONE model the whole time.

Ideal routing (if attended, switching per phase): | Work | Model | Effort | |---|---|---| | Phase A (memory), D (gateway), E (scheduler), any architectural fork / stuck debugging | Opus 4.8 | High | | Phase B (cost wiring), C (API), safety wrapper | Sonnet 4.6 | Medium | | Wave 2 agents F1–F4 (mirror `data_analyst.py`) | Sonnet 4.6 | Medium | | Dashboard wiring (later) | Sonnet 4.6 | Medium (High for tricky state) |

Unattended overnight run (the actual plan): `Opus 4.8 + High` the whole way. No human to catch a wrong call → quality/reasoning beats token savings. Accept higher cost as the price of autonomy. Mixed-model economy is for attended work later. (Cost-priority alternative: Sonnet 4.6 High end-to-end.)

Unattended rules: keep a `BLOCKED — needs Gaurav` list, skip human-gated steps, finish everything else, report at wake. Never auto-post, never auto-auth, never enter credentials. Known blockers: F3 ManyChat account, any approval/credential step. (*Ollama blocker removed — grunt = Haiku, no local install.*)

MODEL-SWITCH CHECKPOINTS (attended build — protect Gaurav's weekly Pro budget)

Default the Claude Code session to Sonnet 4.6 (Gaurav is awake → catches errors → Opus's autonomy edge doesn't apply, and Opus burns the weekly allowance severalx faster). Switch UP to Opus 4.8/xhigh **only** for the hard phases below. At each checkpoint the build session **MUST STOP, tell Gaurav exactly which model+effort to set via `/model`, and wait for his "done" before continuing.** Do not silently proceed across a switch.

Checkpoint (before starting)	Set model	Why
Phase A — persistent brain / narrative memory	Opus 4.8 · xhigh	architectural; schema + recall design
Phase B — cost wiring	Sonnet 4.6 · medium	mechanical bug-fix + plumbing
Phase C — expose data via API	Sonnet 4.6 · medium	mechanical endpoint wiring
Phase D — LiteLLM model gateway	Opus 4.8 · xhigh	single source of truth; routing logic
Phase E — scheduler + scrape-cache	Opus 4.8 · high	infra design (Railway worker, cache)
Untrusted-content LAW	Sonnet 4.6 · medium	bounded wrapper, small
Phase G — Brand-Book Report v6	Opus 4.8 · xhigh	heavy reasoning + report quality
Phases H, I, J, K3, L — client-experience wiring	Sonnet 4.6 · medium	mostly mechanical UI/flow wiring
\$4.5 Pilot (paid)	Gaurav-supervised	API-billed, NOT weekly budget; greenlit live
\$4.7 Wave 3 — hardening + code/security review	Opus 4.8 · max	deep reasoning over whole repo; the one place max earns its cost
Any stuck debugging (any phase)	bump to Opus 4.8 · xhigh	quality beats tokens when blocked

Note: this protects the **Claude Code weekly budget** (the build). The agents' own runtime models (Opus/Sonnet per agent) are CODE written in Phase D and billed to the **Anthropic API**, a separate pool.

SECURITY-REVIEW CHECKPOINTS (hacker-proofing — run mid-build, not just at the end)

Security is gated at every point a new attack surface lands. At each gate the build session **MUST STOP, tell Gaurav to set `/model` to Opus 4.8 · max** (security reasoning wants max effort), run `/security-review` over the new surface, **fix every High/Critical before continuing**, then **tell Gaurav to set the model back** to the next phase's setting. Log Medium/Low for Gaurav. No silent switches.

Security gate (after this lands)	Set model	Attack surface to prove closed
SG1 — after Phase C (API endpoints)	Opus 4.8 · max	authz on every <code>/api/*</code> , brand-scoped RLS actually enforced, input validation, no secrets/tokens in responses
SG2 — after the untrusted-content LAW	Opus 4.8 · max	every external datum wrapped DATA-not-instruction; no bypass path; prompt-injection contained
SG3 — after Phase I (uploads)	Opus 4.8 · max	file-upload: type/size validation, path-traversal safe, malware-safe handling; cloud-link fetch = SSRF-safe (no internal-network/metadata fetch); stored outside web root

Security gate (after this lands)	Set model	Attack surface to prove closed
SG4 — after Phase J (concierge chat)	Opus 4.8 · max	client chat can't trigger un-gated actions; prompt-injection via chat can't escalate; tiered actions can't bypass the approval gate
SG5 — Wave 3 final pass	Opus 4.8 · max	full-repo <code>/security-review</code> + ultra code-review (see §4.7) before front-end

Hacker-proof baseline (apply everywhere, verified at each gate): no secrets in code/responses · all input validated + output encoded · RLS multi-tenant isolation re-tested (no cross-brand reads) · approval gate has no bypass (no auto-post/auto-auth/credential path) · untrusted content always wrapped before the model · rate-limit public endpoints · cloud-link fetches SSRF-guarded · fail-closed on tool/permission errors.

3. WAVE 1 — FOUNDATIONS (the 10-day UC1 core)

Legend: ● exists · ● wire · ● new

Phase A — Persistent Brain (narrative memory) ●

- **A1** Audit `brand_memory` schema; reuse if it fits, else add `brand_narrative : (id, brand_slug, ts, agent, entry_type[decision|action|result], summary, refs jsonb)` . Inherits RLS.
- **A2** `base_agent.py` : `narrative_read(brand,n)` at session_start + `narrative_append(brand,entry)` at session_end.
- **A3** `orchestrator.py` : pass story-so-far into agent context so runs **continue**, not cold-start.
- **A4** Layer 2: `embedding vector` column + pgvector similarity recall. Ship A1–A3 first.
- *Note:* this replaces Monday's `_record_learning` loop — growth agents append here.

Phase B — Cost wiring (kills ₹0 bug) ●

- **B1** Fix `GRID_RUN_ID` propagation in `dashboard_api._run_agent_subprocess` .
- **B2** `cost_tracker` / `tracing` → write per-run cost into `agent_runs` / `usage_logs` .
- **B3** Verify each run STORES cost; widgets stop reading ₹0.

Phase C — Expose real data via API (backend layer for the future cockpit) ●

- **C1** Endpoints return real run/narrative/cost/metric data (the data the cockpit will bind to).
- **C2** Approve/Change/Publish + Allocate/Queue endpoints wired to the existing approval gate + publishers.
- **C3** (*Front-end deferred*) the new cockpit UI is rebuilt later via Emergent/Lovable (`docs/DASHBOARD_DESIGN_EXPLORATION.md`); Wave 1 only guarantees the API/data is real and ready.

Phase D — Model gateway (LiteLLM) ● (= today's D ∪ Monday M7#4)

- **D1** `agents/model_gateway.py` wrapping LiteLLM; config `agent → model + effort` . Single source of truth (kills the current drift between `agents/*.py` MODEL constants and `managed_agents/registry.json`).
- **D2** Route `base_agent` LLM calls through it; per-call cost feeds Phase B.
- **D3** Two Claude tiers + pure-math. **NO Haiku, NO Ollama** (decided Jun 9 — Sonnet 4.6 is the floor):
- **Opus 4.8** (creative/decisions): ceo-brain (xhigh), strategy (high), script-writer (high), **creative-director (medium)**, ad-strategist (high), brand-guardian (high).
- **Sonnet 4.6** (floor — everything else with an LLM, effort medium): content-planner, carousel-designer, data-analyst, funnel, trend-researcher, website, seo, email, community-manager, dm-hunter.
- **None (\$0 pure-math):** trend-sentinel, performance-tracker. `cost_tracker` → make pure-math/none (no LLM).
- **FAL** = media gen (images/video), unchanged.
- Migrate all `claude-opus-4-6` → `claude-opus-4-8` (free, no breaking change).
- *Cost note:* Sonnet floor means high-volume grunt (dm-hunter scoring, community categorization, trend clustering) runs ~3× a Haiku tier. Accepted for quality consistency; revisit only if it bites.

Phase E — Scheduler + scrape-cache ●

- **E1** APScheduler running as a **dedicated 24×7 Railway worker service** (separate from the web API) that triggers scheduled agent runs server-side (daily/twice-daily cadence). **Replaces the old Mac-awake + local crontab** (GRIDLOCK-AUTOPOST-04JUN) — no Mac dependency. Needs Railway hobby (~\$5/mo, always-on; not a sleeping free tier). Feeds the morning brief.

- **E2** Scraping fetcher (anti-bot, MCP) as Apify alternative for owned scrapes; **cache** → many posts per scrape (~80% less re-scrape bleed). Apify for hardest targets only.
- **E3** Trend hashtags → official **IG Hashtag Search API** (saves Apify cost + no ban risk). DM-Hunter prospecting stays on Apify (Hashtag Search strips post-owner identity). Wire each to the right source.

Cross-cutting LAW (fold in now) — Untrusted-content boundary 🟡 (Monday M7#1)

- `agents/_untrusted.py` : `untrusted_context_message(label, content)` + policy preamble. Every external datum (comments, scraped profiles, emails, DMs) wrapped **DATA-not-instruction** before hitting the model. Critical for Wave 2 agents + hosted/client safety. Effort S.

Wave-1 sequencing: A→B→C make it real for a client; D→E built behind a working app; the law lands with C/Wave 2.

3.5 WAVE 1.5 — CLIENT-EXPERIENCE & BRAND-BOOK LAYER (FLOW v2)

From `docs/FLOW_V2_DECISIONS.md` + `docs/BRAND_BOOK_REPORT_SPEC.md`. This is the layer the pilot exercises. **P0** = required to run the 2-brand pilot end-to-end · **P1** = after pilot validated. Legend: 🟢 exists · 🟡 wire · 🟡 new

Phase G — Brand-Book Report v6 🟡 (P0 — the pilot's headline artifact)

Rebuild the report generator to `BRAND_BOOK_REPORT_SPEC.md` (existing V5 is strong → evolve, don't rebuild). - **G1** 8-part architecture: 0 Cover/Scorecard · **1 BRAND FOUNDATION** (new) · 2 Where-You-Stand + benchmark · 3 Market · 4 Content-Intel · 5 Audience-Intel · 6 Growth-Playbook · 7 Horizon · Appendix+provenance. - **G2 Part 1 Foundation** = positioning stmt, value prop, 3–5 messaging pillars + proof, voice & tone (do/don't + vocabulary), ICP/personas, 90-day north-star. **Onboarding:** client signs off → writes `brand_profile.json` + `voice_profile.json`. **Sellable:** "recommended foundation." - **G3 Full-category benchmarking** (not just 3 named): category avg + median, **percentile rank**, **Share-of-Voice %**, leaderboard rank-out-of-N. (Upgrades V5's leaderboard.) - **G4 Real audience demographics** via IG Graph Insights (age/gender/city/active-hrs) for the onboarding/connected version; inferred for the cold sellable version. - **G5 Provenance** per metric (REAL scraped/Insights vs AI-ESTIMATED), Rule-10 `data_provenance`. - **G6 Reliability:** template contract (no Part without its real data fields) + AutoResearch 3-variant + **eval rubric** (every section cites ≥1 real number; benchmark present; brand-voice/no-fabrication check; Foundation complete; no AI-filler). Render Playwright HTML→PDF — **force white bg** (dark-mode bug). - **G7 Two-use config flag:** `cold_sellable` vs `onboarding_connected` (depth per the spec table).

Phase H — Onboarding flow + Brand-Book SIGN-OFF gate (Step 3.5) 🟡 (P0)

- **H1** After research, auto-generate the Brand-Book (Phase G) → client review (approve / request-change) on portal, PDF export. **Hard gate:** strategy/calendar do NOT run until Foundation is approved & green (each pipeline step proceeds only when the prior is complete + green — Gaurav's dependency-chain rule).
- **H2** On approval: write Foundation → `brand_profile.json` + `voice_profile.json` + append `brand_narrative`.

Phase I — Upload surfaces 🟡 (I-a P0 · I-b P0 for askgauravai founder video)

- **I-a Brand-asset ingestion** (onboarding + persistent): accept **(a) cloud link** (Google Drive/Dropbox) and **(b) direct file/folder/PDF/image** upload. **Auto-pull** past posts from connected accounts (no manual upload of what's already on-platform). Store `brands/{slug}/assets/` (or Supabase storage); read by brand-guardian / creative-director / strategy.
- **I-b Per-content-card production upload:** when script-writer flags "founder video/voice required," that card gets an upload slot (file or cloud link) → routes to creative-director for edit → back to approval.

Phase J — Concierge chat (Chief of Staff router) 🟡 (P0 basic)

Single interface (build on existing **The Brain** ⚡; = Phase C team-room trigger). Client talks to ONE agent, never the 6 directly. **Tiered:** trivial/deterministic (reschedule, pause, swap slide, caption edit) = execute instantly, **no LLM spin-up**; substantive (re-plan, inject a trend, new angle) = dispatch the right specialist → result lands in the **approval dashboard**. Available pre- AND post-approval (client is never boxed in).

Phase K — Guardrails 🟡 (K1/K3 P0 · K2 P1-paid)

- **K1 Approval-everywhere (no trust dial):** nothing publishes/uploads without explicit approval. (Auto-rules are LEARNED later by running our own brands, then codified — not shipped now.)
- **K2 Ad-spend gate:** ad-strategist gets a budget cap + per-spend approval, **separate from the content gate** (spending real ad money ≠ posting). P1 — pilot is organic-first.

- **K3 Change discipline:** track every change request; **revision cap**; flag when a "change" is really new scope (a Phase-2 upsell) — anti scope-creep.

Phase L — Notifications 🟡 (P0 minimal — email · WhatsApp P1)

Ping the client when something needs approval (email first; WhatsApp later). This is what keeps "approve-everything" fast without a trust dial. Drives the morning brief + the "Needs you" queue.

Phase M — CRM (Atomic CRM on Supabase) 🟡 (P1 — after pilot core)

Adopt [marmelab/atomic-crm](#) (React + shadcn + Supabase = our exact stack, MIT). Loop-A leads (from F2/F4) land here as the pipeline's exit. Also the resold client-CRM ("ours at extra cost" when client has none). Twenty = fallback if outgrown. Reuses our Supabase + RLS, no new heavy service.

Wave-1.5 sequencing: G → H → I run first (they make the pilot possible) · J + L (basic) alongside · K3 with H · K2 + M + WhatsApp after the pilot proves the core.

4. WAVE 2 — PLATFORM GROWTH DEPARTMENT (after foundations)

All real-time data only, zero assumptions, `data_provenance` + `validate_citations`, drafts → `pending_approval` → Notion, never auto-send, append to `brand_narrative`. Mirror `agents/data_analyst.py` pattern + AutoResearch + LOOP header.

- **F1 Community Manager (#13)** → `agents/community_manager.py`. Reads REAL inbound (IG Graph comments, YouTube Data API via `youtube_oauth.py`, LinkedIn/X via `brands/{slug}/inbound/{platform}.json` paste-in/Chrome-MCP). Categorize (purchase_intent/question/positive/negative/spam/prospect) → draft in Gaurav voice → `pending_approval/community-manager/`.
- **F2 Warm DM Hunter (#14)** → `agents/dm_customer_hunter.py`. 3-tier risk ladder (engaged→DM-eligible / hashtag-participants / niche→engage-only). prospect-researcher (ICP 1–10, LinkedIn-weighted) + outreach-writer (value-first, 2–3 variants, real-detail). Warm-up caps. Every prospect has real Apify/Graph `data_provenance`. → `pending_approval/dm-customer-hunter/`.
- **F3 AI Setter (ManyChat + Claude)** (*Monday M8.1; docs/AI_SETTER_SYSTEM.md*) — IG DM sales automation. Keyword comment → ManyChat public reply + private DM → Claude runs qualify/objections/book-call. Setter=inbound; complements F2 (outbound). Gaurav homework: ManyChat account. Build Claude prompt + 4 reply variations.
- **F4 Lead-magnet funnel (Engine B)** → magnet from live 3-mistakes pillar + `funnel_specialist.py` CTA; `website_agent.py` opt-in page → `POST /api/leads/capture` → Supabase `subscribers` table (`id, brand_id=brands(id), email, name, product_interest, source, captured_at, UNIQUE(brand_id,email)`); `agents/email_marketing_agent.py` (#12) nurture via Gmail MCP → `pending_approval/email-marketing-agent/`.

4.5 PILOT VALIDATION — test the rebuilt pipeline on 2 brands

Goal (Gaurav, Jun 9): prove the whole pipeline end-to-end on a **new** brand and an **existing** brand before declaring done. Run AFTER Wave 1 + Wave 1.5 P0 phases land. Each step must go **green** before the next runs (dependency chain). No auto-post / auto-auth — approval gates honored.

Brand 1 — offgrid-creatives-ai = NEW-brand path

Already onboarded (M6) but profile re-anchored to the Reporting SaaS → exercise the *cold/new* pipeline from the top: onboard/confirm profile → research (cached) → **generate Brand-Book v6 (cold_sellable depth)** → sign-off gate → strategy → 30-day calendar → script → creative/carousel → soul-check → approval → publish → engage. **Validates:** new-brand cold path + report generation without prior history.

Brand 2 — askgauravai = EXISTING-brand path

Has `voice_profile.json`, history, live connections, prior content. Run as a *warm/continuing* brand: narrative continuity (Brain reads story-so-far, doesn't cold-start) → **Brand-Book v6 (onboarding_connected depth) with REAL IG Insights demographics** → founder-video upload via per-card slot (I-b) → full pipeline. **Validates:** existing-brand warm path + real-demographics report + narrative continuity + upload flow.

Pilot acceptance criteria (must all pass)

1. Brand-Book v6 generates for BOTH brands with real data + provenance tags, passes the eval rubric (G6).

2. Foundation sign-off writes `brand_profile.json` + `voice_profile.json` and the gate blocks downstream until green.
3. **Cost is stored per run** (₹0 bug dead — Phase B) and shows on the API.
4. `brand_narrative` appends across the run; askgauravai run **continues** context, doesn't restart cold.
5. Every output passes through the approval gate → `pending_approval` → approved → publish. Nothing auto-posts.
6. Concierge handles one trivial change (instant) + one substantive change (dispatch → approval) per brand.
7. askgauravai report shows real Insights demographics; offgrid (cold) shows inferred — both correct per G7.
8. Scheduler triggers a run server-side with no Mac awake (Phase E).

4.7 WAVE 3 — HARDENING, CODE REVIEW & SECURITY (before the front-end)

Goal (Gaurav): after Wave 1 + 1.5 + pilot land, do a max-effort hardening pass so the SYSTEM is tight and error-free BEFORE the new front-end is built on top of it. Runs once, attended. **Model for this whole wave: Opus 4.8 · max** (this is the one place max effort earns its cost — deep reasoning over the full codebase to catch what mechanical building missed).

W3.1 — Per-area hardening sweep (Opus 4.8 / max)

Go area by area (memory, cost, API, gateway, scheduler, untrusted-LAW, brand-book, onboarding/gate, uploads, concierge, guardrails, notifications, Wave-2 agents). For each: re-read the code at max effort, find loose ends (unhandled errors, missing validation, silent failures, ₹0-style regressions, drift between `agents/*.py` and `registry.json`, un-wrapped untrusted input, missing `data_provenance`), **fix**, re-verify. Commit per area.

W3.2 — Deep code review (`/code-review` ultra tier)

Run `/code-review` at the **ultra** tier over the full Wave-1→pilot diff (multi-agent cloud review). Triage findings: real bugs → fix now (Opus 4.8/max); style/altitude → `/simplify`; uncertain → log for Gaurav. Re-run until the diff comes back clean.

W3.3 — Security review (`/security-review`)

Run `/security-review` over the changes AND the standing surfaces that the front-end will expose: - Every `/api/*` endpoint: authz (brand-scoped RLS actually enforced, not just present), input validation, no secrets in responses, no raw token display (Connections rule). - Untrusted-content boundary: confirm every external datum (comments/scrapes/emails/DMs) is wrapped DATA-not-instruction before the model (the cross-cutting LAW) — no bypass paths. - Supabase RLS: re-test multi-tenant isolation (one brand cannot read another's rows). - Approval gate: confirm nothing can publish/spend without passing the gate (no auto-post/auto-auth/credential path). - `pending_approval` / file writes: path-traversal safe, no JSON leaking to chat. Fix every High/Critical before front-end; log Medium/Low for Gaurav.

W3.4 — Verification + eval gate

Run `verification-loop` + `eval-harness` (`docs/EVAL_HARNESS.md`) so each agent still passes its rubric after the fixes. Green here = the system is ready for the new UI. Produce a one-page `WAVE3_HARDENING_REPORT.md` (what was found, what was fixed, what's logged for Gaurav). No raw JSON.

Cost flag: this wave (Opus 4.8/max + ultra code-review across the whole repo) is the **single most token-heavy thing in the plan**. Run it as its own session, area by area — NOT one giant pass — and `/compact` between areas. The `/code-review` ultra tier runs in the cloud (separate from the per-message build flow).

5. PARKED — PHASE 2 SHELF (good, not on the 10-day path)

- **Postiz** (self-host social scheduling, AGPL, separate Hetzner service) — Phase-2 publish layer; `publishing/*` stays fallback. Postiz MCP available if needed.
- **listmonk** — Mailchimp-alt for email (replaces Gmail nurture later).
- **Coolify** — own hosting (Vercel+Railway replacement) at scale.
- **Plausible CE** — privacy analytics for client dashboards (needs ClickHouse).
- **Anthropic-Cybersec-Skills** — pre-client security pass (community repo, selective).
- **M7#5 deep-research pipeline + nh3-sanitized HTML** — upgrades the Reporting Project; adopt nh3 sanitize for rendered output early (safety).
- **M7#7 fail-closed tool gate** (`NON_ADMIN_BLOCKED_TOOLS` , `mcp__*` admin-only) — **before first client login** (UC1 has client sessions).

- **M7 Tier-3 borrows:** verifier subagent (→ eval harness), atomic_io/readiness (degraded-state on health surface), internal-tool loopback token.
 - **Penpot** — SKIP (our creative pipeline is generative, not design-tool-based).
-

6. DROPPED (per new system / already skipped Monday)

- Obsidian-as-agent-memory, ChromaDB local vector (C1/C2 above).
 - MCP manager/OAuth (`src/mcp_manager.py`) — we wire MCP via Claude SDK.
 - Email inbound triage (`src/email_thread_parser.py`) — we're outbound-first.
 - Odysseus megafire style (204KB) — their acknowledged tech debt.
-

7. OPS + GAURAV HOMEWORK

- askgauravai **YouTube token refresh** — re-consent for community/hashtag perms + analytics scope.
 - **ManyChat account** (for F3 AI Setter).
 - **WhatsApp notification channel** (Phase L WhatsApp tier, P1) — decide provider (WhatsApp Business API / Twilio) when we move past email pings.
 - Hetzner account — only if/when Postiz adopted (Phase 2).
 - License note: Odysseus MIT, Tongyi DeepResearch Apache-2.0 → keep NOTICE when adapting. No AGPL/copyleft in core (Postiz stays a *separate service*, code not pulled into core).
-

8. COST REALITY (M8.6)

- Gaurav on **Claude Pro \$20 (not Max)** — Pro = Claude Code; agents use Anthropic API separately.
 - Est: minimal ~\$65–75/mo · full weekly pipeline ~\$110–150 · heavy ~\$170–220.
 - Biggest items: Anthropic API (agents) ~\$30–60 · Pro \$20 · FAL \$5–15 · Railway \$7–10 · ManyChat \$0–15.
 - Levers: Phase D Ollama local (\$0 grunt), Phase E cache + IG Hashtag API (cut Apify), Class-1 agents stay no-LLM.
 - **Atomic CRM (Phase M) reuses our existing Supabase → ~\$0 extra infra.** Brand-Book v6 adds Anthropic API tokens per report (it's a heavy multi-section gen — runs once per brand per cycle, not per post).
-

9. RISKS / OWNED CALLS

- Memory recall quality is on us (no vendor) → relational-first, pgvector second.
- All-4 foundations in 10 days is tight → each lean or the window slips; Wave 2 only after Wave 1.
- Scrapling self-hosted needs proxy/maintenance vs managed Apify → keep Apify fallback.
- Local Ollama can't do brand-voice creative → hybrid is permanent.